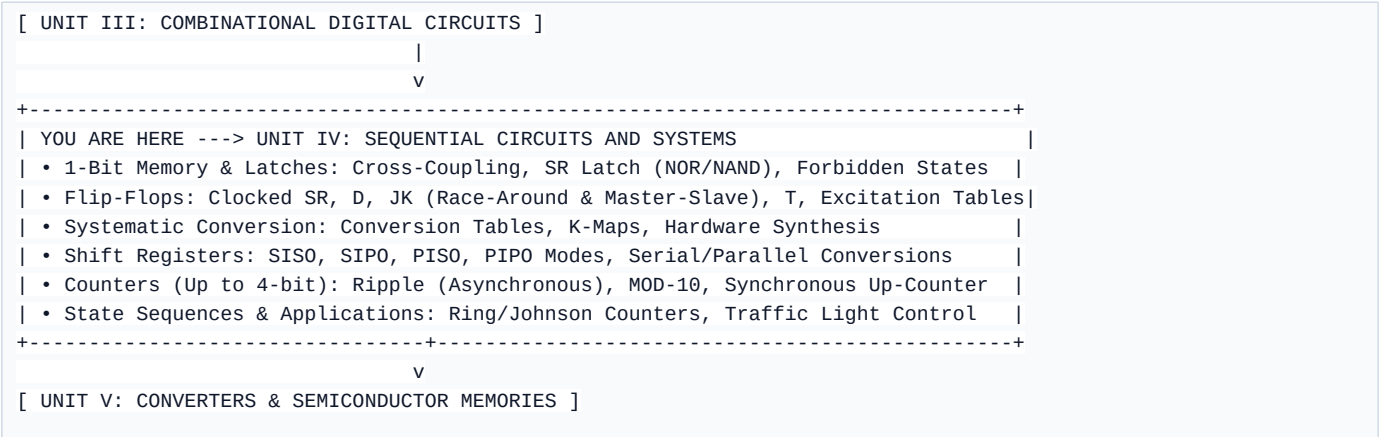


# DEC Unit IV: Sequential Circuits & Systems

## Comprehensive First-Principles Lecture Notes

### Unit IV: Sequential Circuits and Systems

#### Master Subject Map: Unit IV Position



In 3 lines: Combinational circuits are completely amnesiac—they have no memory of the past. Unit IV introduces **feedback**, transforming gates into circuits that store electrical history. We construct 1-bit memory cells (Flip-Flops), pipeline registers (Shift Registers), and rhythmic state machines (Counters) that automate real-world systems like traffic light intersections.

#### Prerequisites for this Unit

Before studying Unit IV, ensure you recall these concepts: 1. **Gate Characteristics & Propagation Delay (Unit I, Section 1.8):** Signals do not change instantaneously; physical gates have a small delay  $t_{pd}$  (typically **5 to 10 ns**). 2. **Universal Gates (Unit I, Section 1.6):** NAND and NOR truth tables. 3. **Pulse Waveforms (Prereq 5):** Rising (positive) edge, falling (negative) edge, pulse width ( $t_w$ ), and clock period ( $T$ ). 4. **Gray Code & State Tables (Unit I, Section 1.4 & Prereq 6):** Reading multi-variable tabular state sequences.

### PART 1: The 1-Bit Memory, Latches, and Clocked Flip-Flops

#### 4.1 Combinational vs. Sequential Logic & The 1-Bit Memory Concept

Source: Kumar (Ch. 10, pp. 577–581, Figures 10.1 & 10.2); Mano (Ch. 5, pp. 248–252)

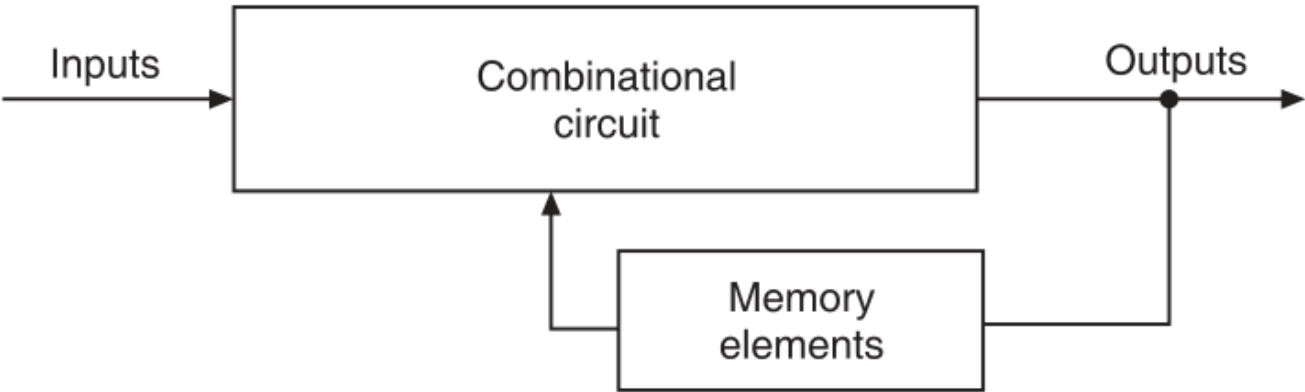
[Pacing: Conceptually Heavy — The Moment Memory is Born]

##### Need (Problem-First)

In an adder or multiplexer, the moment you remove your finger from the input switch, the output disappears. But a real computer must store passwords, remember loop counters, and hold program instructions. How can simple logic gates—which only know how to react to present voltages—be made to **remember a bit permanently**, even after the original input signal has gone to zero?

##### Chain of Cause and Effect

Feed an output wire back into an input terminal (feedback) → the circuit's output now reinforces its own input → the circuit possesses two stable physical states ( $Q=1$  or  $Q=0$ ) → it remains locked in its current state indefinitely until an external trigger forces a flip → a **Bistable Multivibrator (1-bit memory)** is created.



**Figure 10.1** Block diagram of a sequential circuit.

Figure

**10.1:** Fundamental block diagram of a Sequential Circuit consisting of a combinational logic network and memory feedback elements. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 578 (printed p. 546).

- Definition of Sequential Circuit:** A digital circuit whose output at any instant depends not only on the present inputs, but also on the **past history of inputs (the present state of the memory elements)** (Kumar, p. 577).

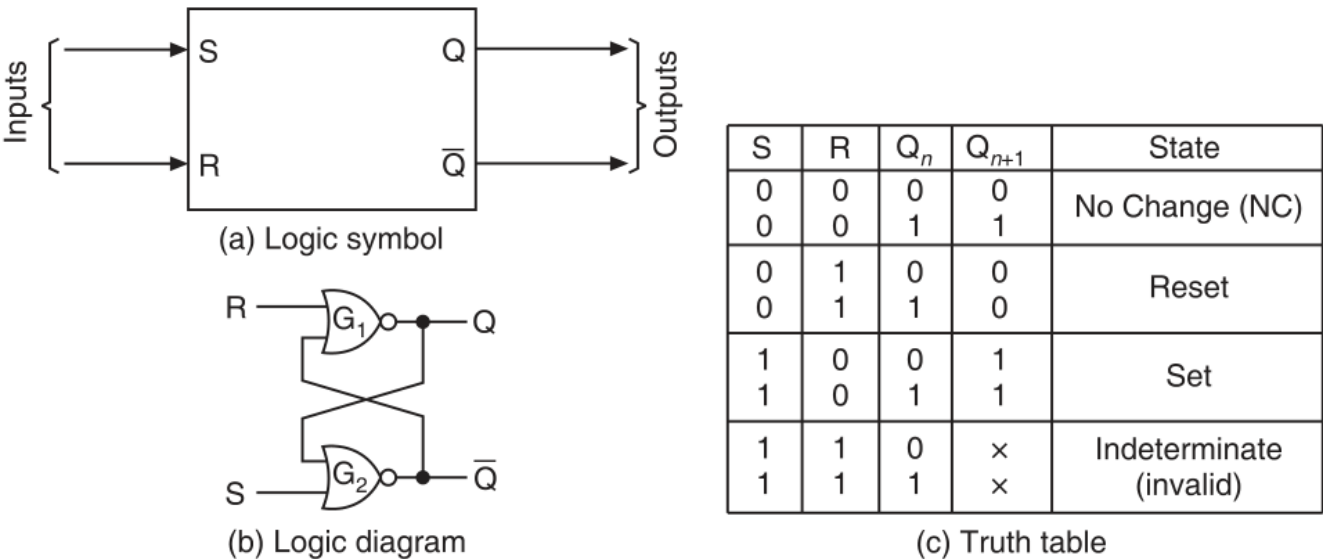
#### 4.2 The Bistable SR Latch: Active-HIGH (NOR) and Active-LOW (NAND)

Source: Kumar (Ch. 10, pp. 581–584, Figures 10.4 & 10.5); Floyd (Ch. 6, pp. 326–332)

[Pacing: Slow & Methodical — Trace the Internal Electron Loops]

##### 1. The Active-HIGH S-R Latch Using NOR Gates (Kumar, p. 581)

Two NOR gates are cross-coupled: the output of Gate 1 ( $Q$ ) is fed into an input of Gate 2, and the output of Gate 2 ( $\bar{Q}$ ) is fed into an input of Gate 1:



**Figure 10.4** Active-HIGH S-R latch.

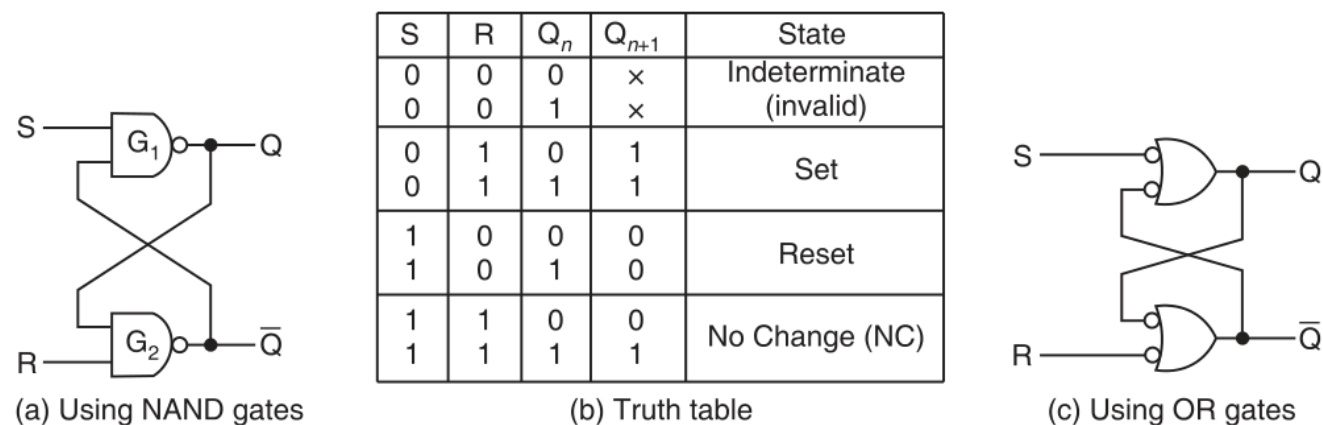
Figure

**10.4:** Active-HIGH Set-Reset (S-R) latch using cross-coupled NOR gates: logic diagram, circuit symbol, and functional truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 582 (printed p. 550).

Recall the fundamental rule of a NOR gate: **any input of 1 forces the output to 0**. 1. **The SET State ( $S = 1, R = 0$ ):** Applying  $S = 1$  to Gate 2 instantly forces its output  $Q = 0$  V. This 0 V travels along the feedback wire into Gate 1. Now Gate 1 has  $R = 0$  and  $Q = 0$ . With both inputs at 0, Gate 1 outputs  $Q = 1$  (+5 V)! Now remove the input ( $S = 0, R = 0$ ): What happens? Output  $Q = 1$  continues feeding into Gate 2, holding  $Q = 0$ , which in turn holds  $Q = 1$ ! **The circuit latched the 1! It remembers that you set it!** 2. **The RESET State ( $S = 0, R = 1$ ):** Applying  $R = 1$  to Gate 1 instantly forces its output  $Q = 0$  V. This 0 V travels to Gate 2. With  $S = 0$  and  $Q = 0$ , Gate 2 outputs  $Q = 1$  (+5 V)! Now remove the input ( $S = 0, R = 0$ ): The feedback holds  $Q = 0$  and  $Q = 1$ . **It remembers that you cleared it!** 3. **The Quiescent / Memory State ( $S = 0, R = 0$ ):** No change. The latch preserves its previous state ( $Q_{\text{next}} = Q_{\text{present}}$ ). 4. **The FORBIDDEN / INVALID State ( $S = 1, R = 1$ ):** If both inputs are driven HIGH simultaneously, both NOR gates are forced to output 0 at the same time:  $Q = 0$  and  $Q = 0$ ! This violates the fundamental mathematical rule that  $Q$  and  $Q$  must be complementary. *The Disaster of Race Condition:* If both inputs now switch back to 0 at the exact same instant, the two gates race against each other to become 1. Whichever gate has a microscopic picosecond advantage in propagation delay wins, leaving the circuit in an **unpredictable, chaotic random state**. Hence,  $S = 1, R = 1$  is strictly forbidden!

## 2. The Active-LOW S-R Latch Using NAND Gates (Kumar, p. 583)

Two NAND gates cross-coupled:



**Figure 10.5** An active-LOW S-R latch.

Figure

10.5: An active-LOW S-R latch using cross-coupled NAND gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 583 (printed p. 551).

- In NAND logic, **any input of 0 forces the output to 1**.
- Setting requires pulling  $S = 0$ .
- Resetting requires pulling  $R = 0$ .
- Resting memory state is  $S = 1, R = 1$ .
- **Forbidden state:**  $S = 0, R = 0$  (both force outputs to 1).

## 4.3 Clocked SR Flip-Flops: Level-Triggering vs. Edge-Triggering

Source: Kumar (Ch. 10, pp. 584–588, Figures 10.7 & 10.10); Floyd (Ch. 6, pp. 332–338)

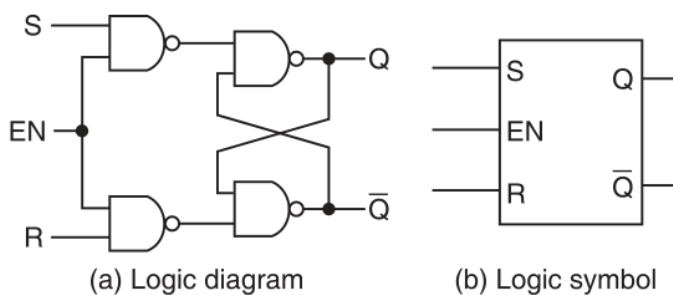
[Pacing: Conceptually Essential — Understanding Synchronization]

### Need (Problem-First)

In an unclocked latch, inputs can change state at any random, chaotic microsecond. In a complex digital processor with millions of gates, signals arriving from different circuit paths experience slightly different wire delays. If latches respond continuously, false temporary glitches will be permanently stored as corrupted data. We need a conductor's baton that commands all memory cells to **update only at precise, synchronized moments in time**. That synchronizing signal is the **Clock (CLK)**.

### The Clocked (Gated) SR Latch (Kumar, p. 584, Figure 10.7)

Two steering NAND gates are placed in front of an active-LOW NAND latch, gated by an Enable / Clock input (**CLK**):



| EN | S | R | $Q_n$ | $Q_{n+1}$ | State                   |
|----|---|---|-------|-----------|-------------------------|
| 1  | 0 | 0 | 0     | 0         | No change (NC)          |
| 1  | 0 | 0 | 1     | 1         |                         |
| 1  | 0 | 1 | 0     | 0         | Reset                   |
| 1  | 0 | 1 | 1     | 0         |                         |
| 1  | 1 | 0 | 0     | 1         | Set                     |
| 1  | 1 | 0 | 1     | 1         |                         |
| 1  | 1 | 1 | 0     | x         | Indeterminate (invalid) |
| 1  | 1 | 1 | 1     | x         |                         |
| 0  | x | x | 0     | 0         | No Change (NC)          |
| 0  | x | x | 1     | 1         |                         |

(c) Truth table

**Figure 10.7** A gated S-R latch.

Figure

**10.7:** A gated (clocked) S-R latch showing input steering gates and enable line. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 584 (printed p. 552).

- When **CLK = 0**: The outputs of both front steering gates are forced to **1** (**S = 1, R = 1**), locking the latch in its resting memory state. The inputs **S** and **R** are completely ignored!
- When **CLK = 1**: The steering gates invert **S** and **R**, allowing them to control the internal latch.

### Level-Triggering vs. Edge-Triggering

Source: Kumar (p. 586); Floyd (pp. 334–336) - **Level-Triggered Latch**: Transparent and sensitive to input changes during the **entire duration that the clock pulse remains HIGH ( $t_w$ )**. If inputs change while clock is HIGH, the output tracks them immediately. - **Edge-Triggered Flip-Flop**: Responds **ONLY during the microscopic transition threshold of the clock edge** (the rising edge **0 → 1** or falling edge **1 → 0**). For the rest of the clock cycle, the flip-flop is completely deaf to input changes. - **Dynamic Indicator Symbol**: A small triangular chevron (**riangleright**) on the clock input terminal indicates **edge-triggering**. A bubble before the chevron indicates **negative (falling) edge-triggering**.

**Setup Time ( $t_s$ ) and Hold Time ( $t_h$ ) Defined Inline (Kumar, p. 599):**

- Setup Time ( $t_s$ )**: The minimum time interval that the digital input data must remain **stable and constant BEFORE the active clock transition edge arrives**. If data changes within  $t_s$ , the flip-flop can enter metastability.
- Hold Time ( $t_h$ )**: The minimum time interval that the input data must remain **stable and constant AFTER the active clock transition edge has passed**.

## 4.4 The JK Flip-Flop, Race-Around Condition, and Master-Slave Architecture

Source: Kumar (Ch. 10, pp. 590–607, Figures 10.17, 10.35, 10.40); Floyd (Ch. 6, pp. 340–348)

**[Pacing: Conceptually Heavy — A Classic Engineering Failure & Its Cure]**

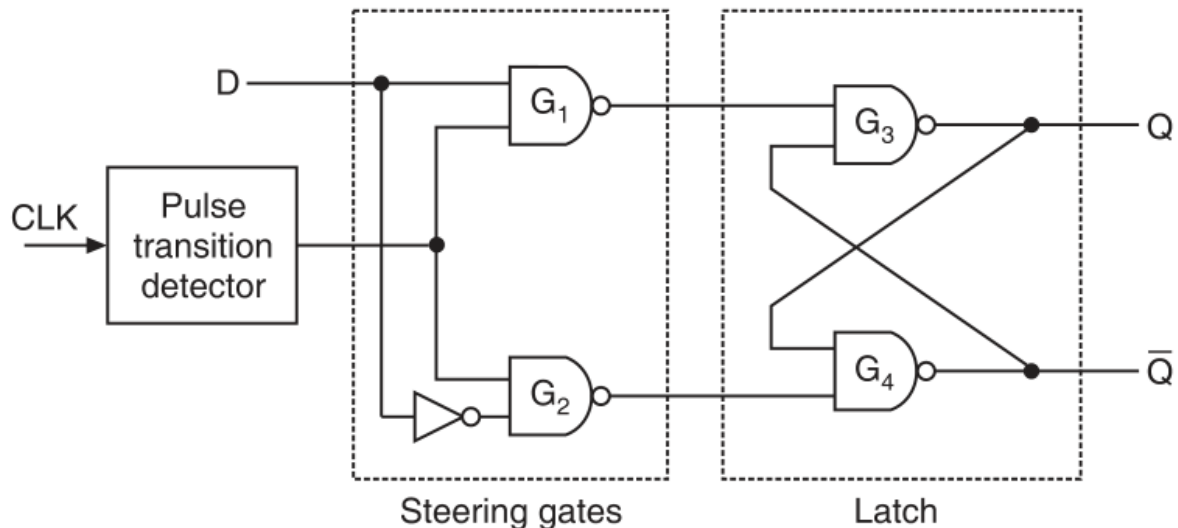
### Need (Problem-First)

The SR flip-flop has a fatal design flaw: the input combination **S=1, R=1** is strictly forbidden because it produces race conditions. In practical engineering, having an illegal input combination that can crash your system is dangerous. We need a flip-flop that eliminates the forbidden state forever, converting it into a useful operation. That device is the **JK Flip-Flop** (named in honor of Jack Kilby, inventor of the integrated circuit).

### The Edge-Triggered JK Flip-Flop (Kumar, p. 590, Figure 10.17)

The outputs **Q** and **Q** are fed back into the front steering gates: - **J** behaves like SET (**S**). - **K** behaves like RESET (**R**). - When **J = 1** and **K = 1**: The feedback causes the circuit to **TOGGLE (invert its state)**! If **Q** was **0**, it flips to **1**; if **Q** was **1**, it flips to **0**. There are **no forbidden states**!

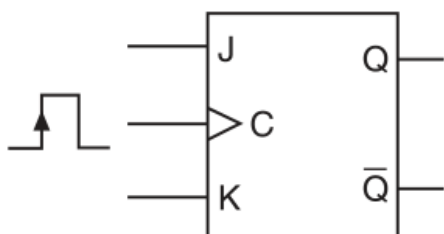
triggered a flip-flop.



**Figure 10.16** Simplified circuit diagram of the edge-triggered D flip-flop.

negative edge-triggered D flip-flop operates in the same way as the positive edge triggered D flip-flop except that the change of state takes place at the negative-going edge of the clock. In the truth table of the negative edge triggered flip-flop the arrows point down

**-triggered J-K flip-flop:** The J-K flip-flop is very versatile and also the n J and K designations for the synchronous control inputs have no known sig functioning of the J-K flip-flop is identical to that of the S-R flip-flop, excep state like that of the S-R flip-flop. The logic symbol and the truth table fo igered J-K flip-flop are shown in Figure 10.17.



(a) Logic symbol

| C          | J | K | $Q_n$ | $Q_{n+1}$ | State          |
|------------|---|---|-------|-----------|----------------|
| $\uparrow$ | 0 | 0 | 0     | 0         | No Change (NC) |
| $\uparrow$ | 0 | 0 | 1     | 1         |                |
| $\uparrow$ | 0 | 1 | 0     | 0         | Reset          |
| $\uparrow$ | 0 | 1 | 1     | 0         |                |
| $\uparrow$ | 1 | 0 | 0     | 1         | Set            |
| $\uparrow$ | 1 | 0 | 1     | 1         |                |
| $\uparrow$ | 1 | 1 | 0     | 1         | Toggle         |
| $\uparrow$ | 1 | 1 | 1     | 0         |                |
| 0          | x | x | 0     | 0         | No Change (NC) |
| 0          | x | x | 1     | 1         |                |

(b) Truth table

**Figure 10.17** Positive edge-triggered J-K flip-flop.

Characteristic Table of the JK Flip-Flop:

| J | K | Q <sub>n</sub> (Present) | Q <sub>{n+1}</sub> (Next State) | Operational Description   |
|---|---|--------------------------|---------------------------------|---------------------------|
| 0 | 0 | 0                        | 0                               | No Change (Hold / Memory) |
| 0 | 0 | 1                        | 1                               | No Change (Hold / Memory) |
| 0 | 1 | 0                        | 0                               | Reset                     |
| 0 | 1 | 1                        | 0                               | Reset                     |
| 1 | 0 | 0                        | 1                               | Set                       |
| 1 | 0 | 1                        | 1                               | Set                       |
| 1 | 1 | 0                        | 1                               | Toggle (0 → 1)            |
| 1 | 1 | 1                        | 0                               | Toggle (1 → 0)            |

- **Characteristic Equation:** Minimizing the table via K-map yields:

$$Q_{n+1} = JQ_n + KQ_n$$

The Race-Around Condition in Level-Triggered JK Flip-Flops

Source: Kumar (Ch. 10, pp. 600–603); Mano (Ch. 5, pp. 256–258)

[Pacing: Slow & Deliberate — Understand the Physical Failure Mode]

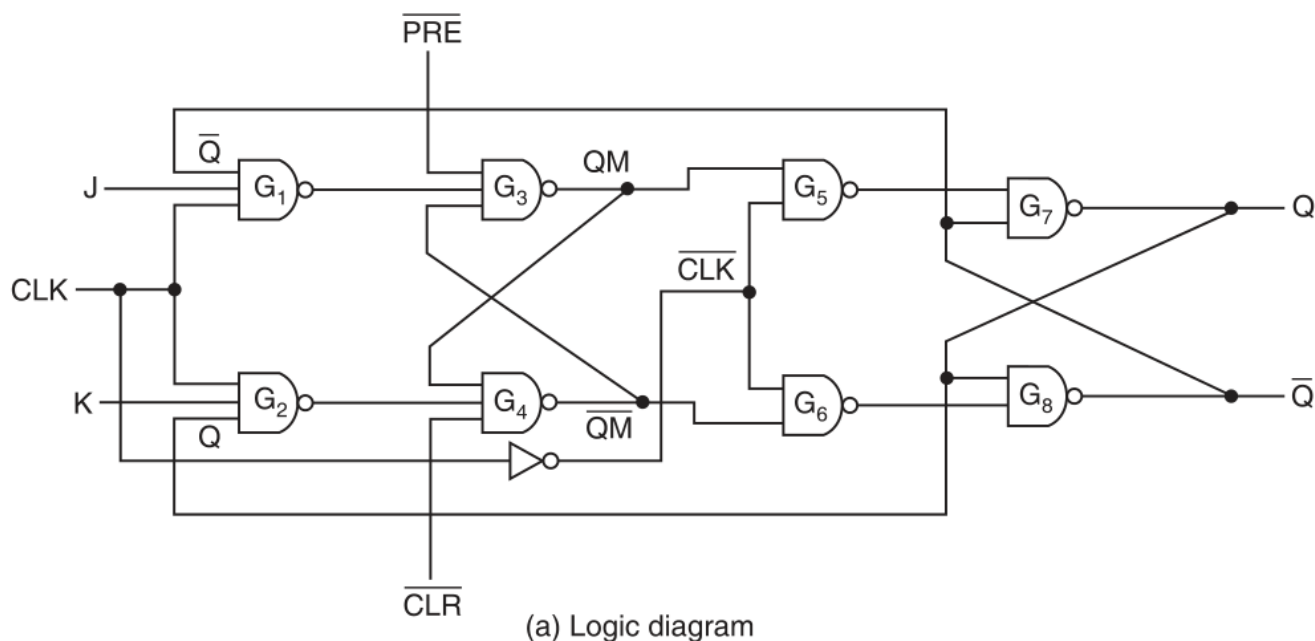
What Physically Goes Wrong:

In a **level-triggered** JK flip-flop, the inputs are active during the entire duration that the clock pulse is HIGH (**t<sub>w</sub>**): - Suppose **J = 1**, **K = 1**, and initially **Q = 0**. - When the clock goes HIGH, the flip-flop toggles to **Q = 1** after its internal gate propagation delay (**t<sub>pd</sub>**). - But if the clock pulse width **t<sub>w</sub>** is longer than the propagation delay (**t<sub>w</sub> > t<sub>pd</sub>**), the new output **Q = 1** feeds right back into the front gates **while the clock is still HIGH!** - This causes the flip-flop to toggle again from **1** back to **0** after another **t<sub>pd</sub>**! - It toggles again to **1**, then back to **0**, oscillating wildly and uncontrollably back and forth during the entire time the clock is HIGH! - When the clock finally falls LOW, the final state of **Q** is completely unpredictable! - This catastrophic oscillation is the **Race-Around Condition** (Kumar, p. 600). - **Physical Condition for Race-Around:**

$$t_w \geq t_{pd}$$

The Engineering Solutions to Race-Around:

1. **Edge-Triggering:** Make clock transitions extremely fast with internal spike detectors so the active window is shorter than **t<sub>pd</sub>**.
2. **The Master-Slave JK Flip-Flop Architecture (Kumar, p. 605, Figure 10.40):**  
Construct the circuit using two separate latches in series: a **Master flip-flop** and a **Slave flip-flop**, driven by complementary clock signals via an inverter:



| Inputs |   |   | Output      | Comments  |
|--------|---|---|-------------|-----------|
| J      | K | C | Q           |           |
| 0      | 0 |   | $Q_0$       | No change |
| 0      | 1 |   | 0           | RESET     |
| 1      | 0 |   | 1           | SET       |
| 1      | 1 |   | $\bar{Q}_0$ | Toggle    |

(b) Truth table

**Figure 10.40** The master-slave J-K flip-flop.

Figure

**10.40:** The Master-Slave J-K flip-flop architecture completely eliminating race-around conditions. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 606 (printed p. 574).

#### How Master-Slave Eliminates Race-Around:

- **Phase 1 (CLK = 1):** The Master flip-flop is enabled and accepts inputs **J** and **K**. But the Slave flip-flop receives **CLK = 0** through the inverter, so the Slave is **completely disabled and locked!** The final outputs **Q** and **Q** cannot change, so no feedback can reach the Master! Race-around is physically impossible!
- **Phase 2 (CLK = 0):** The Master is disabled, locking its internal state. The Slave receives **CLK = 1** and transfers the Master's state to the final outputs **Q** and **Q**.
- The two stages never conduct simultaneously, isolating input from output and destroying the race path.

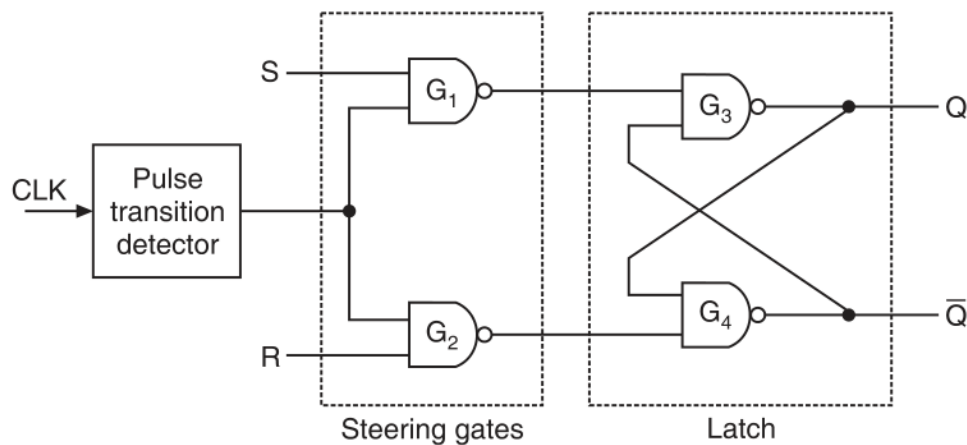
## 4.5 The D (Data) and T (Toggle) Flip-Flops

Source: Kumar (Ch. 10, pp. 589–594, Figures 10.15 & 10.20); Floyd (Ch. 6, pp. 338–344)

[Pacing: Simple & Direct — The Two Workhorses of Registers & Counters]

### 1. The D Flip-Flop (Data / Delay Flip-Flop) (Kumar, p. 589)

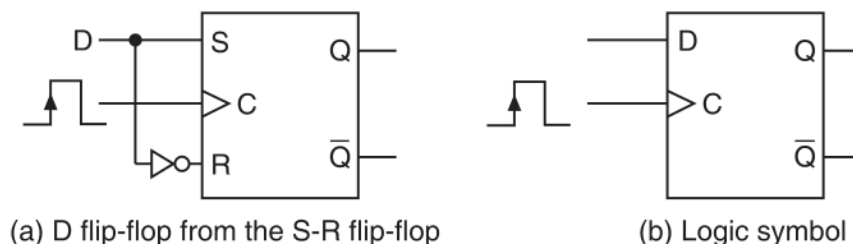
- **Need:** Eliminate the possibility of invalid states in an SR latch by forcing the two inputs to always be exact complements of each other (**S = D**, **R = D**).
- **Mechanism:** A single input **D** is fed directly to the SET line, and through an inverter to the RESET line:



**Figure 10.14** Simplified circuit diagram of the edge-triggered S-R flip-flop.

The edge detector generates a positive spike at the positive-going or negative-going clock pulse. The steering gates 'direct' or 'steer' the narrow spike either to G<sub>1</sub> or G<sub>2</sub> on the state of the S and R inputs.

**Edge-triggered D flip-flop:** The edge-triggered D flip-flop has only one input. A D flip-flop may be obtained from an S-R flip-flop by just putting one inverter between the D input and the S and R inputs (Figure 10.15a). Figures 10.15b and c show the logic symbol and the truth table for the edge-triggered D flip-flop. Note that, this flip-flop has only one synchronous input, the clock. This is called the D (data) input. The operation of the D flip-flop is as follows. The output Q will go to the same state that is present on the D input at the positive-going edge of the clock pulse. In other words, the level present at D will be stored in the flip-flop at the positive-going transition occurs. That is, if D is a 1 and the clock is applied, Q goes to a 1 and  $\bar{Q}$  to a 0 at the rising edge of the pulse and thereafter remain so. If D is a 0 and the clock is applied, Q goes to a 0 and  $\bar{Q}$  to a 1 at the rising edge of the clock pulse and thereafter remain so.



| C          | D | $Q_n$ | $Q_{n+1}$ | State          |
|------------|---|-------|-----------|----------------|
| $\uparrow$ | 0 | 0     | 0         | Reset          |
| $\uparrow$ | 0 | 1     | 0         |                |
| $\uparrow$ | 1 | 0     | 1         | Set            |
| $\uparrow$ | 1 | 1     | 1         |                |
| 0          | x | 0     | 0         | No Change (NC) |
| 0          | x | 1     | 1         |                |

(c) Truth table

**Figure 10.15** The positive edge-triggered D flip-flop.

**Figure 10.15:** Positive

- **Operation:** Whatever bit is on the **D** input pin is captured and transferred to output **Q** on the active clock edge:

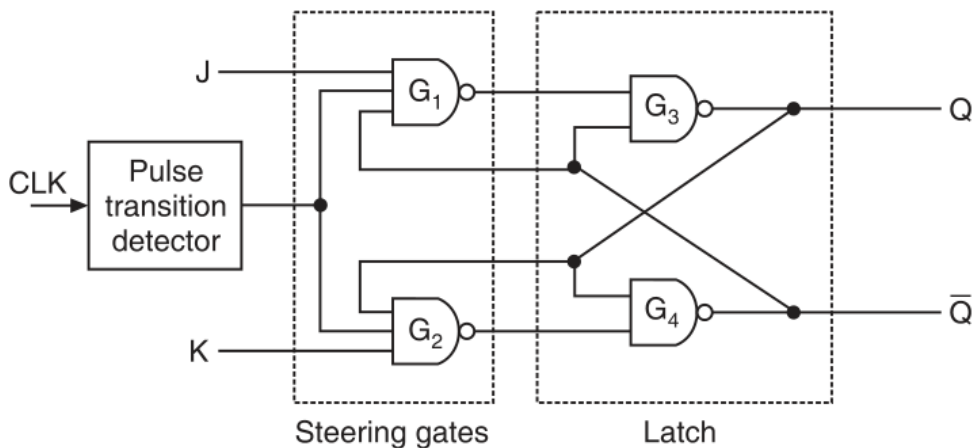
$$Q_{n+1} = D$$

- It introduces a delay of exactly one clock period, making it the universal building block for **registers and computer RAM**.

## 2. The T Flip-Flop (Toggle Flip-Flop) (Kumar, p. 592)

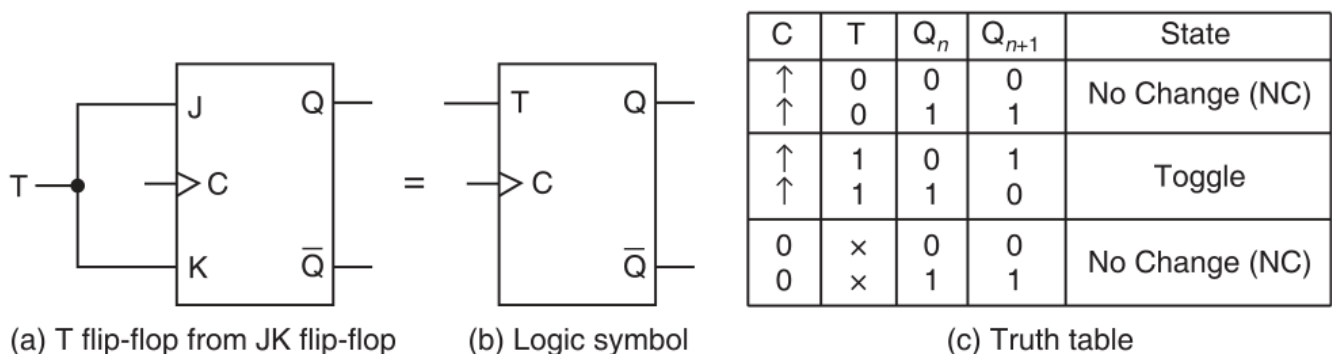
- **Need:** A 1-bit memory cell that flips its state on every clock pulse when enabled, performing automatic **frequency division** ( $f_{out} = f_{in}/2$ ) and binary counting.
- **Mechanism:** Formed by tying the **J** and **K** inputs of a JK flip-flop together (**J = K = T**):

anges its state.



**Figure 10.19** Simplified circuit diagram of the edge-triggered J-K flip-flop.

**edge-triggered T flip-flop:** A T flip-flop has a single control input, labelled T for toggle. When T is HIGH, the flip-flop toggles on every new clock pulse. When T is LOW, the flip-flop remains in whatever state it was before. Although T flip-flops are not widely available commercially, it is easy to convert a J-K flip-flop to the functional equivalent of a T flip-flop by just connecting J and K together and labelling the common connection as T. Thus, when T = 1, we have J = K = 1, and the flip-flop toggles. When T = 0, we have J = K = 0, and so there is no change of state. The logic symbol and the truth table of a T flip-flop are shown in Figure 10.20.



**Figure 10.20** Edge-triggered T flip-flop.

Figure

10.20: Edge-triggered T flip-flop logic diagram, graphic symbol, and characteristic table. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 592 (printed p. 560).

- **Operation:**
- If **T = 0**:  $Q_{n+1} = Q_n$  (Hold / Memory).
- If **T = 1**:  $Q_{n+1} = \overline{Q_n}$  (Toggle / Invert).
- **Characteristic Equation:**

$$Q_{n+1} = T \oplus Q_n = TQ_n + \overline{T}\overline{Q}_n$$

## PART 2: Excitation Tables and Flip-Flop Conversions

### 4.6 Excitation Tables: The Inverse Design View

Source: Kumar (Ch. 10, pp. 593–595); Mano (Ch. 5, pp. 268–270)

[Pacing: Conceptually Essential for Counter & State Machine Design]

#### Need (Problem-First)

In circuit *analysis*, you know the inputs (**J**, **K** or **S**, **R**) and you calculate what the next output  $Q_{n+1}$  will be. But in circuit *synthesis* (designing a counter or traffic light controller), you face the exact opposite problem: **you know what present state the circuit is currently in ( $Q_n$ )**, and you know what state you **WANT** it to become ( $Q_{n+1}$ ) on the next clock pulse. What **input voltages must you apply to force that transition?**

The tabular ledger answering this question is the **Excitation Table**.

Master Compilation of Excitation Tables (Kumar, pp. 593–594):

| Desired Transition ( $Q_n \rightarrow Q_{n+1}$ ) | S-R Inputs ( <b>S</b> , <b>R</b> ) | J-K Inputs ( <b>J</b> , <b>K</b> ) | D Input ( <b>D</b> ) | T Input ( <b>T</b> ) |
|--------------------------------------------------|------------------------------------|------------------------------------|----------------------|----------------------|
| <b>0</b> $\rightarrow$ <b>0</b> (Hold LOW)       | <b>0</b> , <b>X</b>                | <b>0</b> , <b>X</b>                | <b>0</b>             | <b>0</b>             |
| <b>0</b> $\rightarrow$ <b>1</b> (Set HIGH)       | <b>1</b> , <b>0</b>                | <b>1</b> , <b>X</b>                | <b>1</b>             | <b>1</b>             |
| <b>1</b> $\rightarrow$ <b>0</b> (Reset LOW)      | <b>0</b> , <b>1</b>                | <b>X</b> , <b>1</b>                | <b>0</b>             | <b>1</b>             |
| <b>1</b> $\rightarrow$ <b>1</b> (Hold HIGH)      | <b>X</b> , <b>0</b>                | <b>X</b> , <b>0</b>                | <b>1</b>             | <b>0</b>             |

(Where **X** represents a Don't Care condition).

#### Why the X Appears in JK Excitation:

- Look at the transition **0**  $\rightarrow$  **0**: We can either command a HOLD (**J=0**, **K=0**) or command a RESET (**J=0**, **K=1**). In both cases, **J** must be **0**, but **K** can be either **0** or **1**! Hence, **J = 0**, **K = X**. This freedom makes JK flip-flops yield significantly simpler combinational driving logic than other types.

### 4.7 Systematic Flip-Flop Conversion Procedure

Source: Kumar (Ch. 10, pp. 610–615, Figures 10.43–10.50)

[Pacing: Systematic 4-Step Engineering Procedure — Practice the Method]

#### Need (Problem-First)

In the laboratory, your design requires a JK flip-flop, but your company's component inventory only has SR flip-flop chips in stock. How can you wrap external combinational logic around an SR flip-flop so it behaves exactly like a JK flip-flop?

The Universal 4-Step Conversion Procedure (Kumar, p. 610):

- Identify Given vs. Desired:** Name the available flip-flop (Given) and the target flip-flop to be synthesized (Desired).
- Construct the Conversion Table:** Combine the characteristic table of the Desired flip-flop with the excitation table of the Given flip-flop.
- K-Map Minimization:** Plot the required inputs of the Given flip-flop as functions of the Desired inputs and present state  $Q_n$ .
- Draw the Logic Diagram:** Wire the derived combinational logic to the input terminals of the Given flip-flop.

Fully-Worked Example: Convert an S-R Flip-Flop to a J-K Flip-Flop (Kumar, p. 611, Figure 10.44)

- Step 1:** Desired = JK; Given = SR.

• Step 2 (Conversion Table):

| Inputs (J, K) | Present State (Q <sub>n</sub> ) | Next State (Q <sub>{n+1}</sub> ) | Required S Input | Required R Input | Transition Reason |
|---------------|---------------------------------|----------------------------------|------------------|------------------|-------------------|
| 0, 0          | 0                               | 0                                | 0                | X                | Hold 0            |
| 0, 0          | 1                               | 1                                | X                | 0                | Hold 1            |
| 0, 1          | 0                               | 0                                | 0                | X                | Reset             |
| 0, 1          | 1                               | 0                                | 0                | 1                | Reset             |
| 1, 0          | 0                               | 1                                | 1                | 0                | Set               |
| 1, 0          | 1                               | 1                                | X                | 0                | Set               |
| 1, 1          | 0                               | 1                                | 1                | 0                | Toggle (0 → 1)    |
| 1, 1          | 1                               | 0                                | 0                | 1                | Toggle (1 → 0)    |

- Step 3 (K-Map Minimization):
- For S:  $S(J, K, Q_n) = \sum m(4, 6) + d(1, 5)$ .  
Grouping minterms 4 and 6 with don't cares yields:

S = JQ<sub>n</sub>

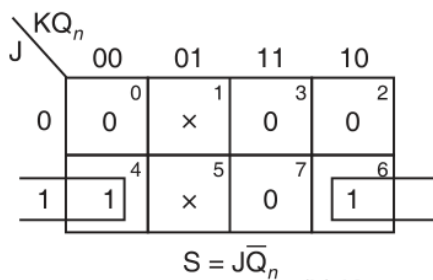
- For R:  $R(J, K, Q_n) = \sum m(3, 7) + d(0, 2)$ .  
Grouping minterms 3 and 7 with don't cares yields:

R = KQ<sub>n</sub>

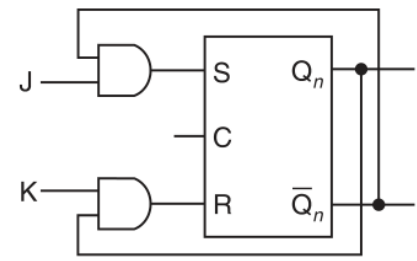
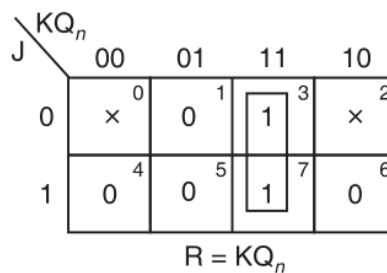
- Step 4 (Logic Schematic Realization): Connect an AND gate receiving J and Q<sub>n</sub> to input S, and connect an AND gate receiving K and Q<sub>n</sub> to input R:

| External Inputs |   | Present State | Next State | Flip-flop Inputs |   |
|-----------------|---|---------------|------------|------------------|---|
| J               | K | $Q_n$         | $Q_{n+1}$  | S                | R |
| 0               | 0 | 0             | 0          | 0                | x |
| 0               | 0 | 1             | 1          | x                | 0 |
| 0               | 1 | 0             | 0          | 0                | x |
| 0               | 1 | 1             | 0          | 0                | 1 |
| 1               | 0 | 0             | 1          | 1                | 0 |
| 1               | 0 | 1             | 1          | x                | 0 |
| 1               | 1 | 0             | 1          | 1                | 0 |
| 1               | 1 | 1             | 0          | 0                | 1 |

(a) Conversion table



(b) K-maps for S and R



(c) Logic diagram

**Figure 10.44** Conversion of S-R flip-flop to J-K flip-flop.

Figure

**10.44:** Hardware conversion of an S-R flip-flop into a J-K flip-flop using two external AND gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 611 (printed p. 579).

## PART 3: Shift Registers (SISO, SIPO, PISO, PIPO) and Applications

### 4.8 Shift Register Operational Modes

Source: Kumar (Ch. 11, pp. 636–643, Figures 11.4–11.9); Floyd (Ch. 7, pp. 389–408)

[Pacing: Physical Data Motion — Trace the Clock Shifts]

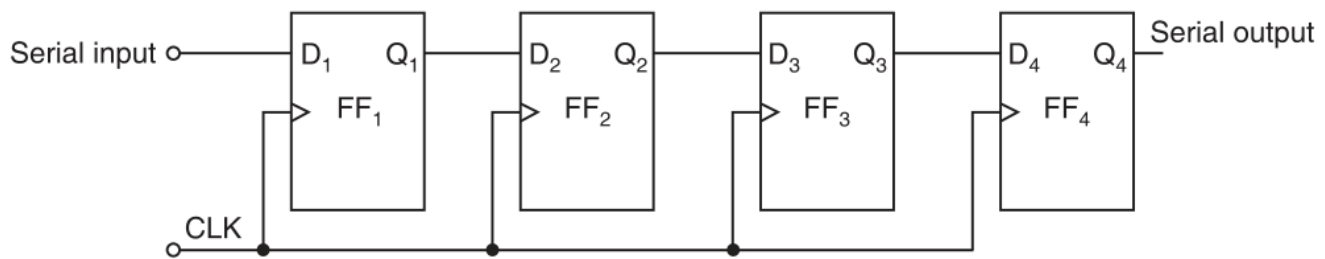
#### Need (Problem-First)

A single flip-flop stores only one bit. To store a word (like a 32-bit integer or an 8-bit ASCII character), we must assemble multiple flip-flops into a **Register**. Furthermore, communication channels (like a USB cable or Wi-Fi radio) transmit data **serially along a single wire**, while computer processors manipulate data **in parallel across 32 or 64 simultaneous bus wires**. We need circuits that can convert serial streams into parallel words and vice versa. These are **Shift Registers**.

#### The Four Fundamental Modes Defined:

##### 1. Serial-In Serial-Out (SISO) (Kumar, p. 640, Figure 11.4)

- **Mechanism:** Data bits enter one at a time on a single wire and exit one at a time from the last stage.
- To store an  $n$ -bit word requires  $n$  clock pulses. To retrieve it completely requires another  $n$  clock pulses (total  $2n$  pulses).



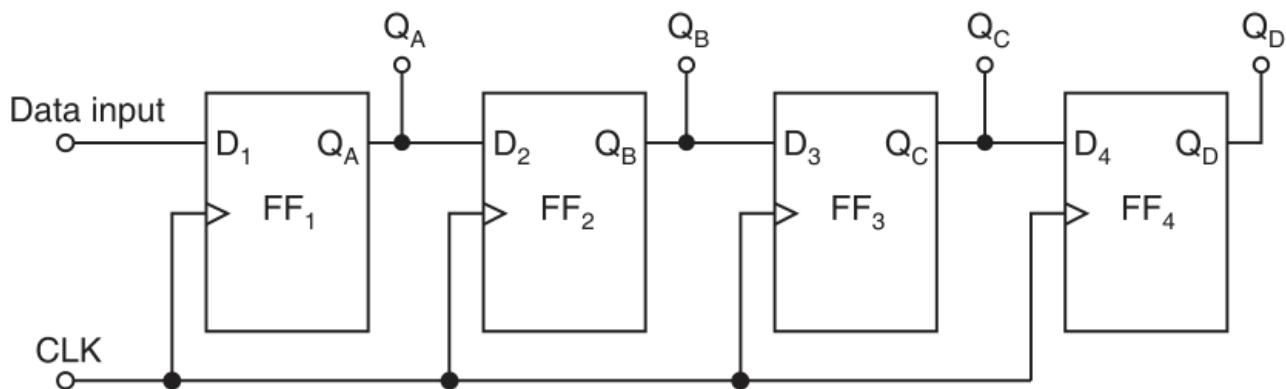
**Figure 11.4** 4-bit serial-in, serial-out, shift-right, shift register.

Figure

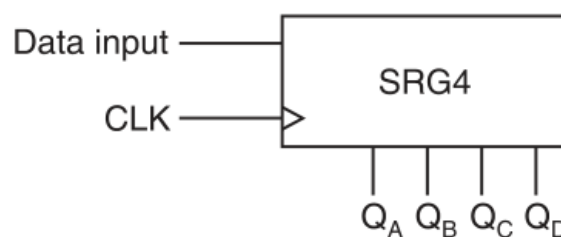
11.4: Logic diagram of a 4-bit Serial-In Serial-Out (SISO) shift register. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 640 (printed p. 608).

## 2. Serial-In Parallel-Out (SIPO) (Kumar, p. 641, Figure 11.7)

- **Mechanism:** Data enters serially along one input pin. Once all bits have been shifted in ( $n$  clock pulses), all flip-flop outputs ( $Q_0$ ,  $Q_1$ ,  $Q_2$ ,  $Q_3$ ) are read **simultaneously in parallel**.
- Essential for: **UART serial receivers** and **USB-to-parallel decoders**.



(a) Logic diagram



(b) Logic symbol

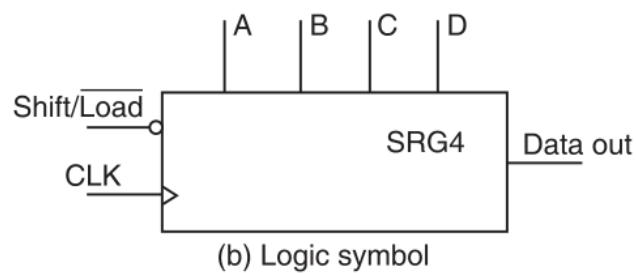
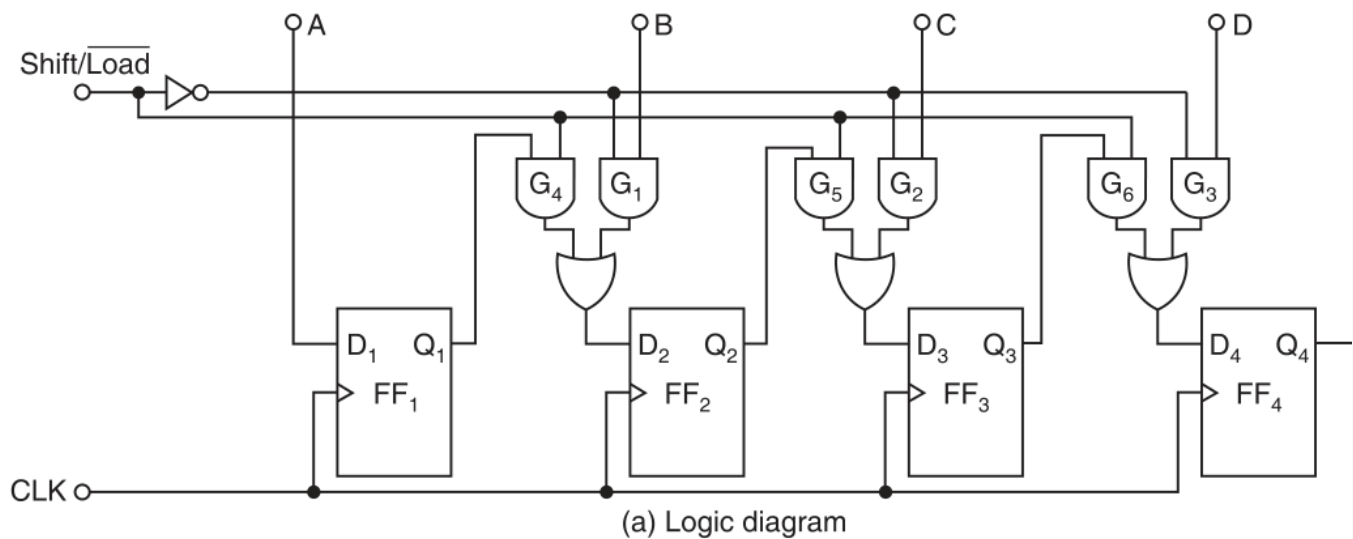
**Figure 11.7** A 4-bit serial-in, parallel-out, shift register.

Figure

11.7: Logic diagram of a 4-bit Serial-In Parallel-Out (SIPO) shift register. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 641 (printed p. 609).

## 3. Parallel-In Serial-Out (PISO) (Kumar, p. 642, Figure 11.8)

- **Mechanism:** An entire multi-bit word is loaded into all flip-flops simultaneously in **one clock pulse** using a **SHIFT/over Line{LOAD}** control line. The bits are then clocked out serially one by one along the output wire.
- Essential for: **Microprocessor serial transmitters**.



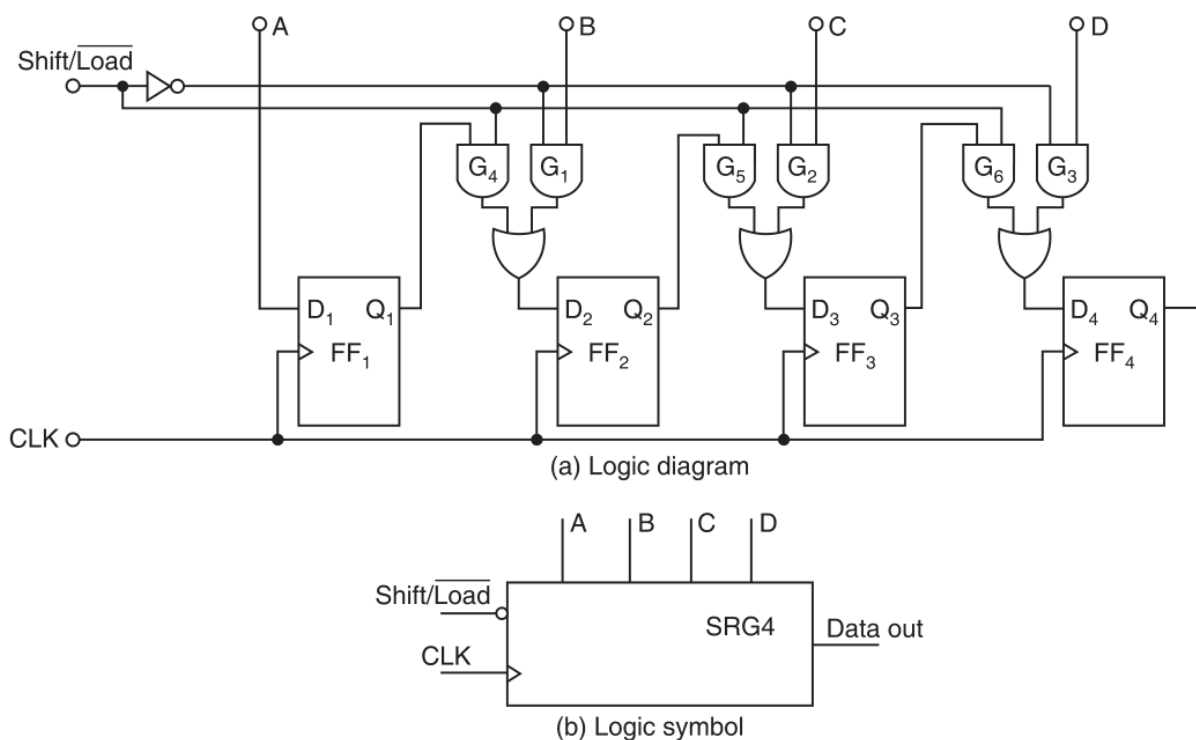
**Figure 11.8** A 4-bit parallel-in, serial-out, shift register.

Figure

**11.8:** Logic diagram of a 4-bit Parallel-In Serial-Out (PISO) shift register with Shift/Load steering gates. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 642 (printed p. 610).

#### 4. Parallel-In Parallel-Out (PIPO) (Kumar, p. 642, Figure 11.9)

- **Mechanism:** Data is loaded simultaneously in **one clock pulse** and read out simultaneously on the next clock pulse.
- Introduces zero serial delay; functions as a temporary **storage buffer register** inside CPUs.



**Figure 11.8** A 4-bit parallel-in, serial-out, shift register.

## 11.8 PARALLEL-IN, PARALLEL-OUT, SHIFT REGISTER

In a parallel-in, parallel-out, shift register, the data is entered into the register in parallel form, and also the data is taken out of the register in parallel form. Immediately following the simultaneous entry of all data bits, the bits appear on the parallel outputs.

Figure 11.9 shows a 4-bit parallel-in, parallel-out, shift register using D FFs. Data is applied to the D input terminals of the FFs. When a clock pulse is applied, at the positive-going edge of that pulse, the D inputs are shifted into the Q outputs of the FFs. The register now stores the data. The stored data is available instantaneously for shifting out in parallel form.

Figure

11.9: Logic diagram of a 4-bit Parallel-In Parallel-Out (PIPO) buffer register. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p.

642 (printed p. 610).

## PART 4: Binary Counters (Ripple & Synchronous) and Traffic Light Control

### 4.9 Asynchronous (Ripple) Counters (Up to 4 Bits)

Source: Kumar (Ch. 12, pp. 662–671, Figures 12.1 & 12.8); Floyd (Ch. 8, pp. 440–446, Figure 11)

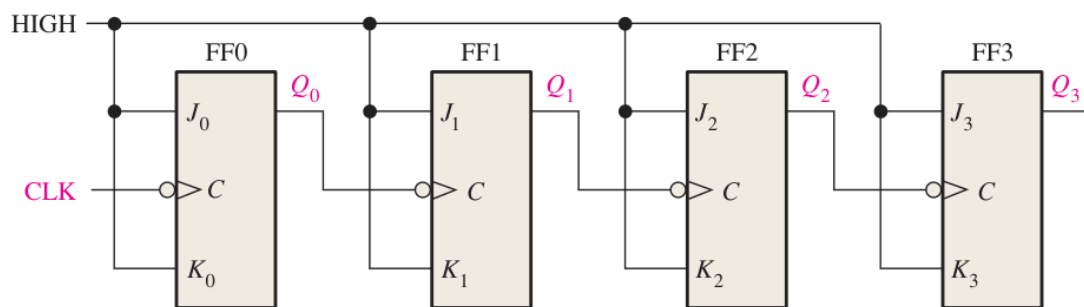
[Pacing: Essential Counter Foundations — Trace the Clock Propagation Delay]

#### Need (Problem-First)

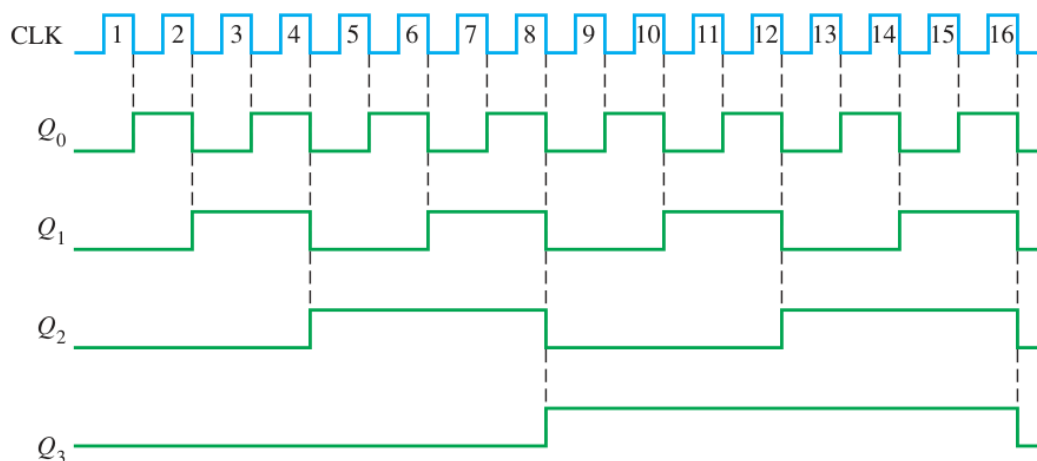
How does a digital system count physical events (e.g., number of cars entering a garage, seconds ticking on a clock, or clock cycles in a program)? We need a circuit that cycles through a sequence of binary numbers upon receiving incoming electrical pulses.

#### 1. The 4-Bit Binary Ripple Up-Counter (Floyd, p. 444, Figure 11)

Four toggle flip-flops (T or JK with  $J=K=1$ ) are cascaded: - **The Asynchronous Wiring Rule:** The external clock is connected **ONLY to the first flip-flop (FF<sub>0</sub>)!** - Each subsequent flip-flop receives its clock input **from the output of the preceding stage (Q<sub>{i-1}</sub>)!**



(a)



(b)

**FIGURE 11** Four-bit asynchronous binary counter and its timing diagram. Open file F08-11 and verify the operation.

\*Answers are at the end of the chapter.

MULTISIM



Figure

11: Schematic diagram and timing waveforms of a 4-bit asynchronous (ripple) binary up-counter. Sourced from Thomas L. Floyd, Digital Fundamentals: A Systems Approach (1st Ed.), p. 444 (printed p. 438).

### Characteristic 13 Walkthrough: Hardware Reality Check

1. Because flip-flops trigger on **negative (falling) clock edges** (1 → 0), FF\_1 toggles only when Q\_0 transitions from 1 to 0.
2. This creates automatic frequency division: Q\_0 toggles every 1 clock cycle ( $f/2$ ), Q\_1 toggles every 2 cycles ( $f/4$ ), Q\_2 every 4 cycles ( $f/8$ ), and Q\_3 every 8 cycles ( $f/16$ ).
3. The outputs Q\_3 Q\_2 Q\_1 Q\_0 count up in perfect binary sequence: 0000 → 0001 → 0010 → dots → 1111\_2 (0 to 15\_{10}).

### The Fatal Flaw of Ripple Counters: Cumulative Propagation Delay

Source: Kumar (p. 670); Floyd (p. 443)

Because each flip-flop is clocked by the preceding flip-flop, the transition delay ripples through the chain like falling dominoes. In an  $n$ -bit ripple counter:

$$t_{\text{total delay}} = n \times t_{\text{pd}}$$

For a 4-bit counter with  $t_{\text{pd}} = 10 \text{ ns}$ , the MSB changes 40 ns after the clock edge! During that 40 ns window, the outputs pass through false, spurious intermediate counts (glitches), destroying accuracy at high clock frequencies.

## 2. The Modulo-10 (Decade / BCD) Ripple Counter (Kumar, p. 669, Figure 12.8)

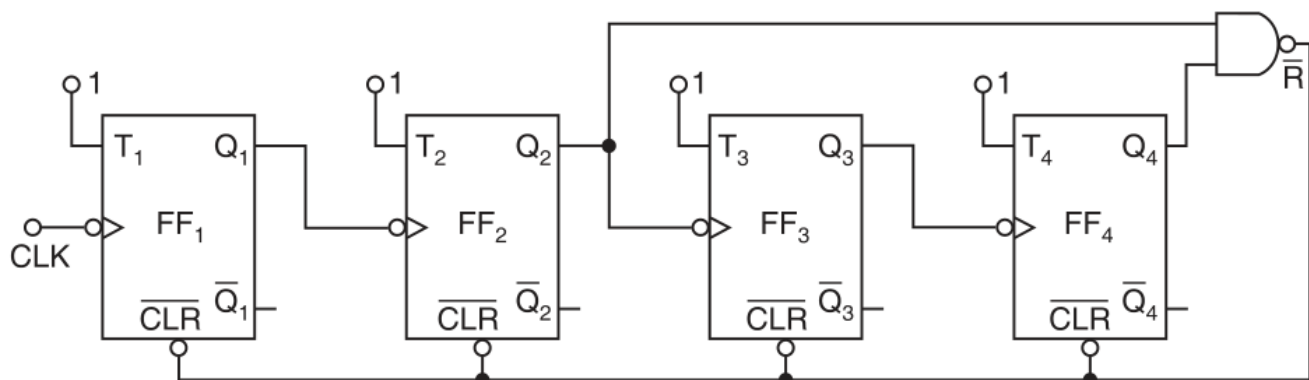
A standard 4-bit counter naturally counts through 16 states (0 to 15, Modulo-16). But decimal clocks and BCD systems must count from 0 to 9 and instantly reset back to 0 on the tenth pulse (Modulo-10). - **The Feedback NAND Reset Mechanism:** Decimal count 10\_{10} in binary is 1010\_2 ( $Q_3=1$ ,  $Q_2=0$ ,  $Q_1=1$ ,  $Q_0=0$ ). Connect lines  $Q_3$  and  $Q_1$  to the inputs of a 2-input NAND gate, and connect the NAND output to the active-LOW Asynchronous Clear (CLR) pins of all four flip-flops:

| After pulses | Count |       |       |       |
|--------------|-------|-------|-------|-------|
|              | $Q_4$ | $Q_3$ | $Q_2$ | $Q_1$ |
| 0            | 0     | 0     | 0     | 0     |
| 1            | 0     | 0     | 0     | 1     |
| 2            | 0     | 0     | 1     | 0     |
| 3            | 0     | 0     | 1     | 1     |
| 4            | 0     | 1     | 0     | 0     |
| 5            | 0     | 1     | 0     | 1     |
| 6            | 0     | 1     | 1     | 0     |
| 7            | 0     | 1     | 1     | 1     |
| 8            | 1     | 0     | 0     | 0     |
| 9            | 1     | 0     | 0     | 1     |
| 10           | 0     | 0     | 0     | 0     |

(a) Count table

| $Q_4Q_3$ \ $Q_2Q_1$ |  | 00 | 01 | 11 | 10 |
|---------------------|--|----|----|----|----|
| 00                  |  | 0  | 1  | 3  | 2  |
| 01                  |  | 4  | 5  | 7  | 6  |
| 11                  |  | 12 | 13 | 15 | 14 |
| 10                  |  | 8  | 9  | 11 | 10 |

(b) K-Map



(c) Logic diagram

**Figure 12.8** Asynchronous mod-10 counter using T flip-flops.

Figure

12.8: Count sequence table, K-map, and logic schematic of an asynchronous Modulo-10 (Decade) counter using feedback clear gating. Sourced from A.

Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 669 (printed p. 637).

- When the counter reaches 9 (1001),  $Q_1$  is 0, so the NAND output remains HIGH (inactive).
- On the next pulse, the counter briefly touches 10 (1010). Instantly, both  $Q_3=1$  and  $Q_1=1$ . The NAND gate output snaps LOW (0 V), pulsing the CLR pins and resetting all flip-flops back to 0000 in just a few nanoseconds!

#### 4.10 Synchronous Counters (Up to 4 Bits)

Source: Kumar (Ch. 12, pp. 674–686, Figure 12.16); Floyd (Ch. 8, pp. 446–451, Figure 19)

[Pacing: Conceptually Critical — The High-Speed Industry Standard]

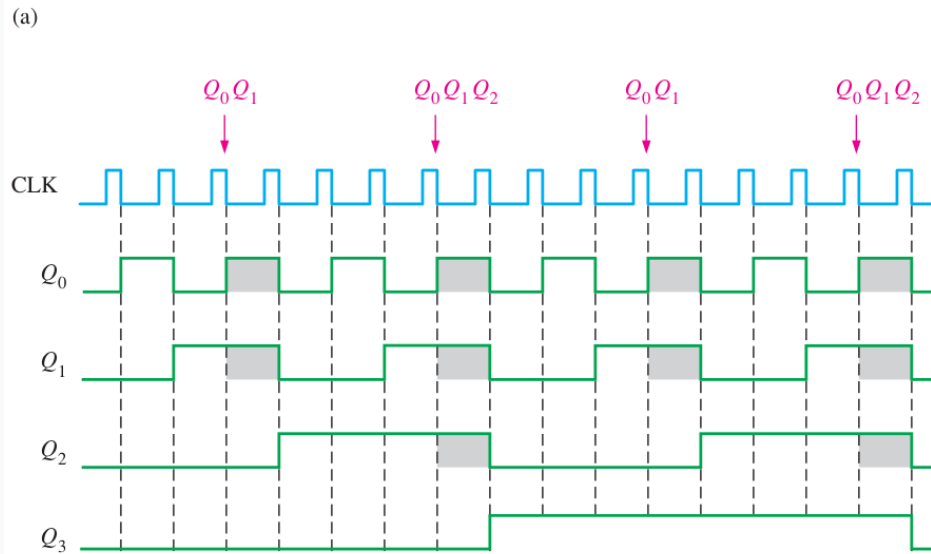
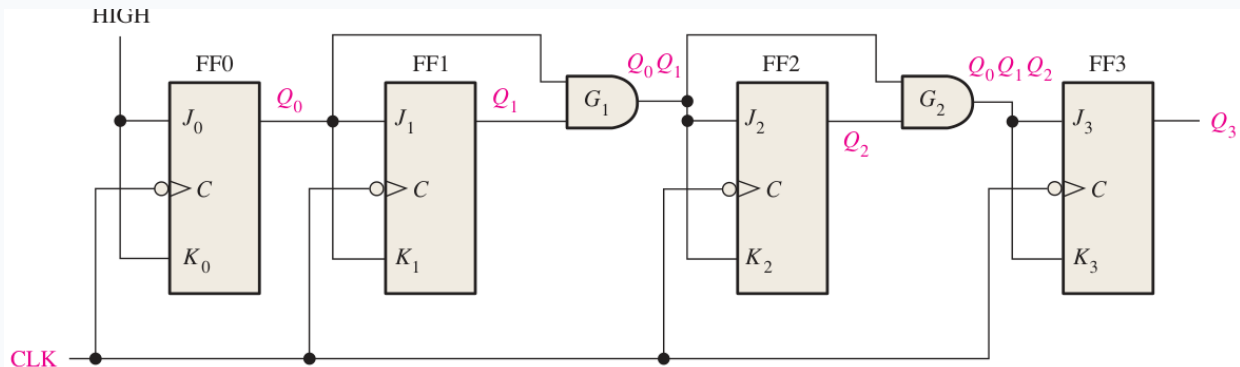
##### Need (Problem-First)

To eliminate the cumulative ripple propagation delay of asynchronous counters, we need a counter where **every single flip-flop is connected to the exact same master clock wire simultaneously**.

##### The Synchronous Mechanism (Floyd, p. 450, Figure 19)

Because all flip-flops receive the clock edge at the exact same picosecond, they must decide *ahead of time* whether they should toggle or stay put. -  $FF_0$  must toggle on **every single clock pulse** ■  $J_0 = K_0 = 1$ . -  $FF_1$  must toggle only when  $Q_0 =$

1 ■  $J_1 = K_1 = Q_0$ . - FF\_2 must toggle only when both  $Q_0 = 1$  AND  $Q_1 = 1$  ■  $J_2 = K_2 = Q_0 Q_1$ . - FF\_3 must toggle only when  $Q_0 = 1, Q_1 = 1, \text{ AND } Q_2 = 1$  ■  $J_3 = K_3 = Q_0 Q_1 Q_2$ .



**FIGURE 19** A 4-bit synchronous binary counter and timing diagram. Points where the AND gate outputs are HIGH are indicated by the shaded areas.

Figure

19: Schematic diagram and timing waveforms of a 4-bit Synchronous Binary Counter showing parallel clock distribution and carry-enable AND gates.

Sourced from Thomas L. Floyd, Digital Fundamentals: A Systems Approach (1st Ed.), p. 450 (printed p. 444).

- **Total Propagation Delay:** Exactly **one flip-flop delay ( $t_{pd}$ )** regardless of how many bits are added! All flip-flops switch simultaneously in unison.

## 4.11 Ring Counters and Johnson (Twisted-Ring) Counters

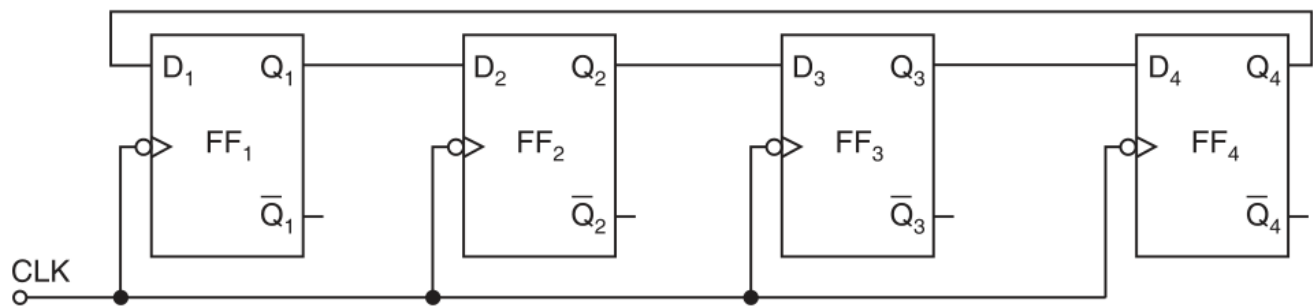
Source: Kumar (Ch. 12, pp. 697–700, Figures 12.57 & 12.61)

[Pacing: Elegant Shift-Register Counter Architectures]

### 1. The 4-Bit Ring Counter (Kumar, p. 697, Figure 12.57)

A 4-bit shift register where the output of the last flip-flop is fed straight back into the input of the first flip-flop ( $D_0 = Q_3$ ): - **Initial Preset:** A single 1 is pre-loaded: 1000. - On successive clock pulses, the 1 simply circulates in a ring:

1000 → 0100 → 0010 → 0001 → 1000. - **Advantage:** Each state directly activates a control line without requiring any decoding gates!



**Figure 12.57** Logic diagram of a 4-bit ring counter using D flip-flops.

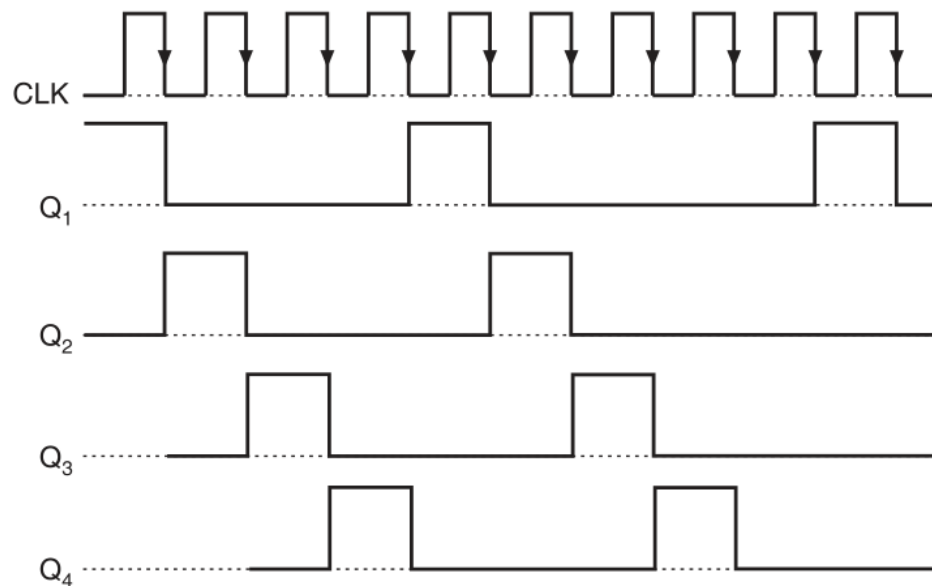
Figure

12.57: Logic diagram of a 4-bit Ring Counter using D flip-flops. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 697 (printed p. 665).

## 2. The 4-Bit Johnson Counter (Twisted-Ring Counter) (Kumar, p. 698, Figure 12.61)

The **inverted** output of the last flip-flop is fed back into the input of the first flip-flop ( $D_0 = \bar{Q}_3$ ): - Starting from 0000, 1s stream in from the left, followed by 0s:

0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000. - An  $n$ -bit Johnson counter yields  $2n$  unique **timing states** (a 4-bit counter gives 8 states!), double that of a basic ring counter, while requiring only simple 2-input AND gates for complete decoding.

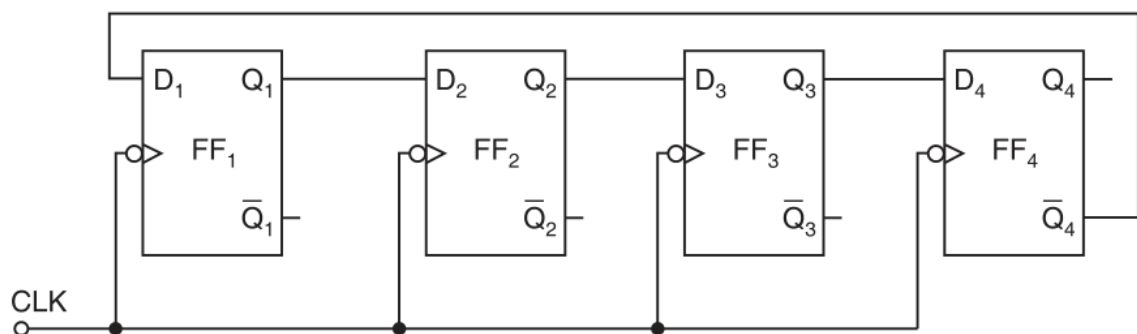


**Figure 12.60** Timing diagram of a 4-bit ring counter.

### 12.9.2 Twisted Ring Counter (Johnson Counter)

This counter is obtained from a serial-in, serial-out shift register by providing feedback inverted output of the last FF to the D input of the first FF. The Q output of each stage is connected to the D input of the next stage, but the  $\bar{Q}$  output of the last stage is connected to the D input of the first stage, therefore, the name *twisted ring counter*. This feedback arrangement produces a sequence of states.

The logic diagram of a 4-bit Johnson counter using D FFs is shown in Figure 12.61. A realization of the same using J-K FFs is shown in Figure 12.62. The state diagram and the table are shown in Figure 12.63. The timing diagram of a Johnson counter is shown in Figure 12.64.



**Figure 12.61** Logic diagram of a 4-bit twisted ring counter using D flip-flops.

Figure

**12.61:** Logic diagram of a 4-bit twisted ring (Johnson) counter. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 698 (printed p.

666).

## 4.12 Real-World Application: Traffic Light Control System

Source: Floyd (Ch. 6, pp. 322–326, Figures 1, 2, 3 & 4)

[Pacing: Comprehensive System Capstone — See How Everything Fits Together]

### Need (Problem-First)

We have studied gates, adders, decoders, flip-flops, and counters. How are these individual chips combined to control an actual automated system in the physical world? We analyze the design of an automated **Traffic Signal Controller for a busy Main**

Street intersected by an occasionally used Side Street (Floyd, p. 322).

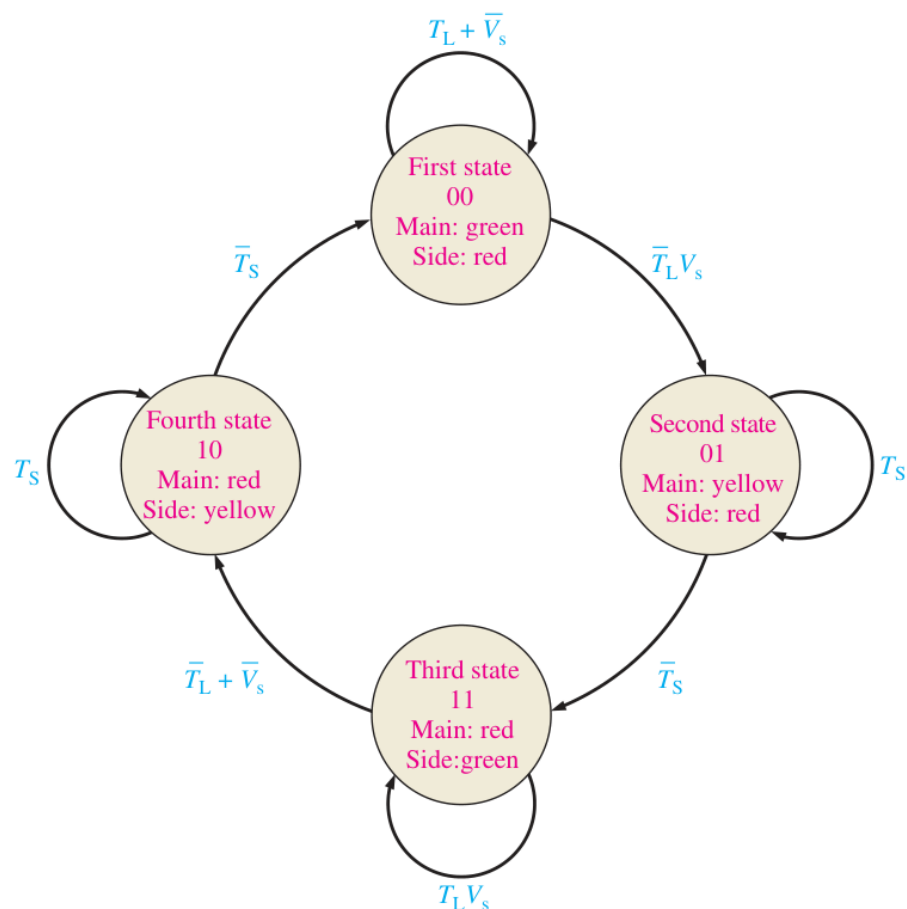
### System Specifications & Operational Requirements:

1. **State 1 (S\_1):** Main street light is **GREEN**; Side street light is **RED**. This state remains active for at least **25 seconds** (Long Timer,  $T_L$ ). If there are no vehicles detected on the side street ( $V_S = 0$ ), the main street stays GREEN indefinitely.
2. **State 2 (S\_2):** When side street traffic arrives ( $V_S = 1$ ) after **25 s**, Main street transitions to **YELLOW** for **4 seconds** (Short Timer,  $T_S$ ); Side street remains RED.
3. **State 3 (S\_3):** Main street is **RED**; Side street turns **GREEN**. This state lasts for **25 seconds** as long as side street vehicles remain present ( $T_L \ V_S$ ).
4. **State 4 (S\_4):** Main street remains **RED**; Side street transitions to **YELLOW** for **4 seconds** ( $T_S$ ). The system then loops back to State 1!

### 1. The State Diagram (Floyd, p. 323, Figure 2)

To eliminate multi-bit race conditions during state transitions, the four states are assigned a **2-bit Gray code sequence**: - State 1 (S\_1): 00 - State 2 (S\_2): 01 - State 3 (S\_3): 11 - State 4 (S\_4): 10

**FIGURE 2** State diagram for the traffic signal control system.

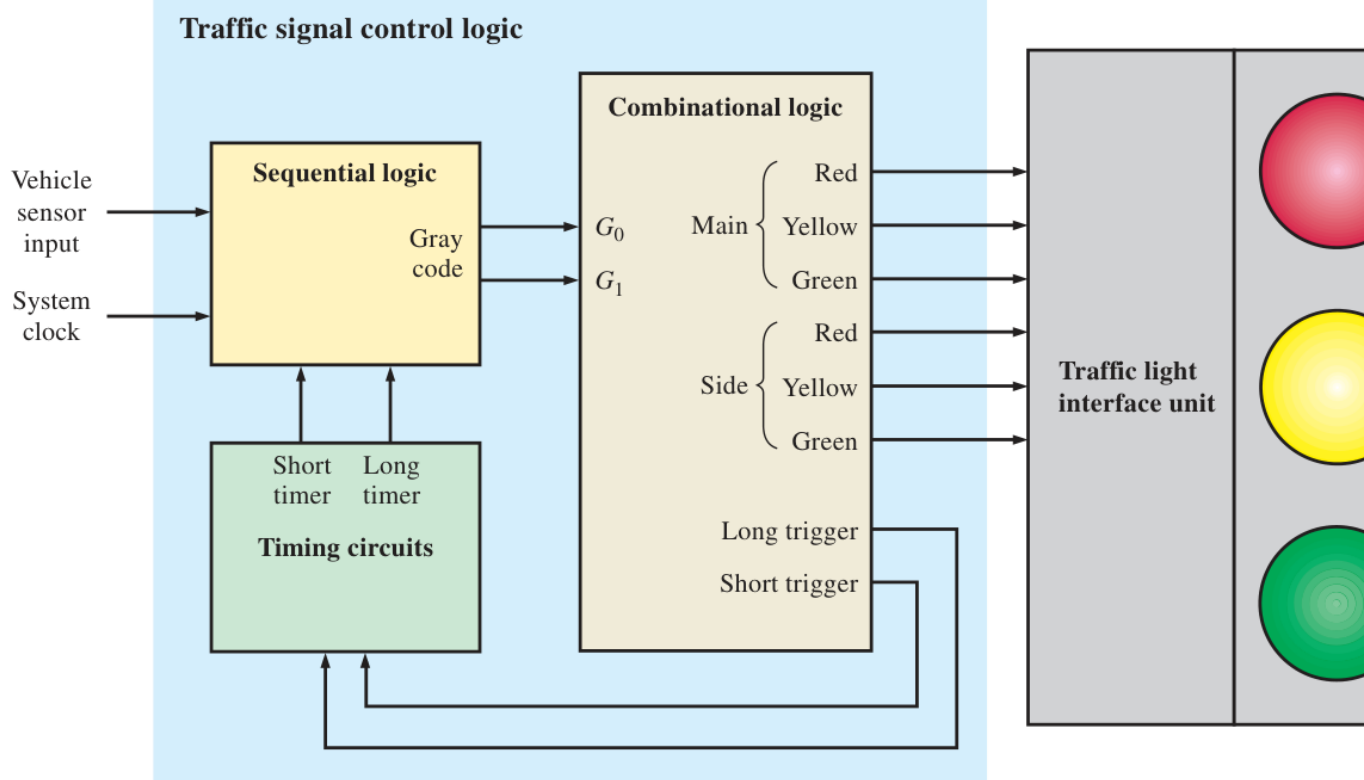


Figure

2: State diagram for the traffic signal controller showing 2-bit Gray code state assignments and transition conditions based on timers ( $T_L$ ,  $T_S$ ) and vehicle sensors ( $V_S$ ). Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 323 (printed p. 317).

### 2. System Architecture & Block Diagram (Floyd, p. 324, Figure 3)

Figure 3 from Floyd reveals the complete three-part system decomposition: 1. **Sequential Logic (The State Counter):** A 2-bit Gray-code counter that sequences through states **00** → **01** → **11** → **10** based on inputs from the vehicle sensor and timers. 2. **Timing Circuits:** A **25 s** long timer ( $T_L$ ) and a **4 s** short timer ( $T_S$ ), triggered by transitions between states. 3. **Combinational Output Logic:** Decodes the 2-bit Gray code into six physical lamp driving signals: Main Red (**MR**), Main Yellow (**MY**), Main Green (**MG**), Side Red (**SR**), Side Yellow (**SY**), and Side Green (**SG**).



**FIGURE 3** Block diagram of the traffic signal control system.

Figure

3: Block diagram of the traffic signal control system showing interaction between sequential logic counter, timing circuits, combinational output decoder, and power interface. Sourced from Thomas L. Floyd, *Digital Fundamentals: A Systems Approach* (1st Ed.), p. 324 (printed p. 318).

#### Derivation of Combinational Light Output Equations (Floyd, p. 325):

- Main Green (**MG**) is ON only in State 1 (**S\_1**, code 00):

$$MG = G_1 G_0$$

- Main Yellow (**MY**) is ON only in State 2 (**S\_2**, code 01):

$$MY = G_1 G_0$$

- Main Red (**MR**) is ON in both State 3 (**S\_3**, 11) and State 4 (**S\_4**, 10):

$$MR = G_1 G_0 + G_1 G_0 = G_1(G_0 + G_0) = G_1$$

- Side Green (**SG**) is ON only in State 3 (**S\_3**, code 11):

$$SG = G_1 G_0$$

- Side Yellow (**SY**) is ON only in State 4 (**S\_4**, code 10):

$$SY = G_1 G_0$$

- Side Red (**SR**) is ON in both State 1 (**S\_1**, 00) and State 2 (**S\_2**, 01):

$$SR = G_1 G_0 + G_1 G_0 = G_1$$

Notice how elegant digital design is: Main Red is simply bit **G\_1**, and Side Red is simply **G\_1**!

## 1. Does This Make Sense? Self-Check

- **Question 1:** Why is a Master-Slave JK flip-flop immune to the race-around condition?
- **Answer:** Because the Master latch and Slave latch are driven by opposite clock phases (**CLK** and **CLK**). When the Master is accepting inputs, the Slave is completely disconnected from the output, breaking the feedback loop.
- **Question 2:** If an input clock signal has a frequency of **16 ext{ MHz}**, what is the frequency at the **Q** output of a single T flip-flop configured with **T=1**?
- **Answer:** Exactly half: **16 ext{ MHz} / 2 = 8 ext{ MHz}**.
- **Question 3:** In a 4-bit SISO shift register, how many total clock pulses are required to load a 4-bit word in and then shift it completely out?
- **Answer:** 4 clock pulses to shift data in, and another 4 clock pulses to shift data out (total 8 clock pulses).

## 2. Conceptual Misconceptions Corrected

- **Misconception:** *"A flip-flop and a latch are the exact same thing."*  
**Correction:** No. A **latch** is level-triggered (transparent during the entire clock HIGH interval). A **flip-flop** is edge-triggered (sensitive only during the infinitesimal picosecond transition edge of the clock).
  - **Misconception:** *"In an asynchronous ripple counter, all flip-flops toggle at the exact same instant."*  
**Correction:** Never! Only the first flip-flop toggles on the clock edge. Each subsequent stage is clocked by the preceding stage, creating a cumulative ripple delay (**n · t\_{pd}**) that can corrupt high-speed systems.
-